

Assignment 3 — Spark vs Ray Distributed Data Pipeline

Vishnu Vardhan
DA24B038

October 5, 2026

1 Overview

Results are at the bottom if you wanna look at them. This document describes the procedure used to implement an identical NYC Yellow Taxi cleaning and transformation pipeline in both **Apache Spark (PySpark 3.5.1)** and **Ray (2.59.0, Ray Data)**, run each on a distributed cluster, and benchmark execution time, resource utilization, and Python UDF overhead.

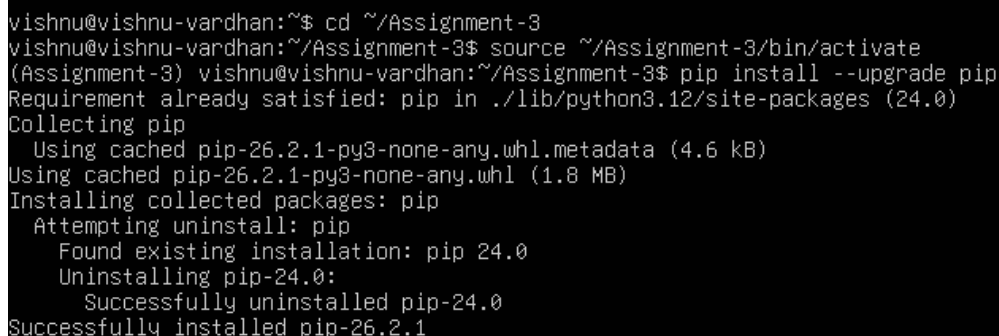
2 Environment Setup

2.1 System Prerequisites

```
sudo apt update
sudo apt install -y openjdk-17-jdk python3-pip python3-venv
```

2.2 Python Virtual Environment

```
python3 -m venv ~/Assignment-3
source ~/Assignment-3/bin/activate
pip install --upgrade pip
```



```
vishnu@vishnu-varadhan:~$ cd ~/Assignment-3
vishnu@vishnu-varadhan:~/Assignment-3$ source ~/Assignment-3/bin/activate
(Assignment-3) vishnu@vishnu-varadhan:~/Assignment-3$ pip install --upgrade pip
Requirement already satisfied: pip in ./lib/python3.12/site-packages (24.0)
Collecting pip
  Using cached pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
Using cached pip-26.2.1-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.2.1
```

Figure 1

2.3 Install Frameworks

```
pip install pyspark==3.5.1
pip install "ray[default]==2.59.0" pandas pyarrow
```

```
(Assignment-3) vishnu@vishnu-varadhan:~/Assignment-3$ python -c 'import pyspark, ray; print("both imported OK")'
both imported OK
(Assignment-3) vishnu@vishnu-varadhan:~/Assignment-3$ java --version
openjdk 17.0.20.1 2026-08-18
OpenJDK Runtime Environment (build 17.0.20.1+1-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.20.1+1-1-24.04-Ubuntu, mixed mode, sharing)
```

Figure 2: Environment test.

2.4 Set SPARK_HOME

```
export SPARK_HOME=$(python -c "import pyspark, os; print(os.path.dirname(pyspark.__file__))")
export PATH=$SPARK_HOME/bin:$SPARK_HOME/sbin:$PATH
echo $SPARK_HOME
```

3 Data Acquisition

```
mkdir -p ~/taxi_data/raw ~/taxi_data/processed
cd ~/taxi_data/raw

wget https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
wget https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

ls -lh
```

Expected output:

```
yellow_tripdata_2024-01.parquet 48M
taxi_zone_lookup.csv 13K
```

```
(Assignment-3) vishnu@vishnu-varadhan:~/Assignment-3$ cd ~/taxi_data/raw
(Assignment-3) vishnu@vishnu-varadhan:~/taxi_data/raw$ wget http://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
--2026-10-04 11:23:30-- http://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
Resolving d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)... 18.67.153.6, 18.67.153.166, 18.67.153.210, ...
Connecting to d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)|18.67.153.6|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet [following]
--2026-10-04 11:23:30-- https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
Connecting to d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)|18.67.153.6|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 49961641 (48M) [binary/octet-stream]
Saving to: 'yellow_tripdata_2024-01.parquet'

yellow_tripdata_2024-01.parquet 100%[=====] 47.65M 3.01MB/s in 50s

2026-10-04 11:24:20 (980 KB/s) - 'yellow_tripdata_2024-01.parquet' saved [49961641/49961641]

(Assignment-3) vishnu@vishnu-varadhan:~/taxi_data/raw$ wget http://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
--2026-10-04 11:26:05-- http://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
Resolving d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)... 99.86.148.201, 99.86.148.124, 99.86.148.12, ...
Connecting to d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)|99.86.148.201|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv [following]
--2026-10-04 11:26:05-- https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
Connecting to d37ci6vzurychx.cloudfront.net (d37ci6vzurychx.cloudfront.net)|99.86.148.201|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12331 (12K) [text/csv]
Saving to: 'taxi_zone_lookup.csv'

taxi_zone_lookup.csv 100%[=====] 12.04K --.-KB/s in 0.002s

2026-10-04 11:26:05 (7.18 MB/s) - 'taxi_zone_lookup.csv' saved [12331/12331]

(Assignment-3) vishnu@vishnu-varadhan:~/taxi_data/raw$ ls -lh
total 48M
-rw-rw-r-- 1 vishnu vishnu 13K Feb 22 2024 taxi_zone_lookup.csv
-rw-rw-r-- 1 vishnu vishnu 48M Mar 21 2024 yellow_tripdata_2024-01.parquet
```

Figure 3: Data Collection

4 Spark Cluster Setup (2 Workers)

4.1 Configuration Files

```
mkdir -p $SPARK_HOME/conf
```

\$SPARK_HOME/conf/spark-env.sh:

```
export SPARK_MASTER_HOST=127.0.0.1
export SPARK_MASTER_PORT=7077
```

```
export SPARK_MASTER_WEBUI_PORT=8080
export SPARK_WORKER_CORES=1
export SPARK_WORKER_MEMORY=512m
export SPARK_WORKER_WEBUI_PORT=8081
export SPARK_DAEMON_MEMORY=100m
```

```
chmod +x $SPARK_HOME/conf/spark-env.sh
```

\$SPARK_HOME/conf/workers:

```
127.0.0.1
127.0.0.1
```

\$SPARK_HOME/conf/spark-defaults.conf:

```
spark.master spark://127.0.0.1:7077
spark.driver.memory 500m
spark.executor.memory 500m
spark.executor.cores 1
spark.sql.shuffle.partitions 4
spark.default.parallelism 2
spark.ui.port 4040
```

4.2 Start the Cluster

```
# Master
cd ~
nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.master.Master \
  --host 127.0.0.1 --port 7077 --webui-port 8080 > spark-master.log 2>&1 &

# Worker 1
nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.worker.Worker \
  spark://127.0.0.1:7077 --webui-port 8081 --cores 1 --memory 512m > spark-w1.log 2>&1 &

# Worker 2
nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.worker.Worker \
  spark://127.0.0.1:7077 --webui-port 8082 --cores 1 --memory 512m > spark-w2.log 2>&1 &
```

4.3 Verify the Cluster

```
jps
```

Expected output:

```
Master
Worker
Worker
Jps
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.master.Master --host 127.0.0.1 --port 7077 --webui-port 8080 &
spark-master.log 2>&1 &
(1) 1929
(Assignment-3) vishnu@vishnu-varadhan:~$ jps
1971 Jps
1929 Master
(Assignment-3) vishnu@vishnu-varadhan:~$ nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.worker.Worker spark://127.0.0.1:7077 --webui-port 8081 --cores
1 --memory 512m > spark-w1.log 2>&1 &
(2) 2000
(Assignment-3) vishnu@vishnu-varadhan:~$ jps
2050 Jps
2000 Worker
1929 Master
(Assignment-3) vishnu@vishnu-varadhan:~$ nohup $SPARK_HOME/bin/spark-class org.apache.spark.deploy.worker.Worker spark://127.0.0.1:7077 --webui-port 8082 --cores
1 --memory 512m > spark-w2.log 2>&1 &
(3) 2009
(Assignment-3) vishnu@vishnu-varadhan:~$ jps
2151 Jps
2000 Worker
2009 Worker
1929 Master
```

Figure 4: adding workers

```
curl -s http://localhost:8080 | grep -iE "Alive Workers|Cores|Status"
```

Expected output:

```
<li><strong>Alive Workers:</strong> 2</li>
<li><strong>Cores in use:</strong> 2 Total,
<li><strong>Status:</strong> ALIVE</li>
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ curl -s http://localhost:8080 | grep -iE "ALIVE|Workers|Cores" | head -10
<li><strong>Alive Workers:</strong> 2</li>
<li><strong>Cores in use:</strong> 2 Total,
<li><strong>Status:</strong> ALIVE</li>
<span class="collapse-aggregated-workers collapse-table" onClick="collapseTable('collapse-aggregated-workers','aggregated-workers')">
  <a>Workers (2)</a>
  <div class="aggregated-workers collapsible-table">
    <thead><th width="" class="">Worker Id</th><th width="" class="">Address</th><th width="" class="">State</th><th width="" class="">Cores</th><th width=""
class="">Memory</th><th width="" class="">Resources</th></thead>
    <td>ALIVE</td>
    <td>ALIVE</td>
    <thead><th width="" class="">Application ID</th><th width="" class="">Name</th><th width="" class="">Cores</th><th width="" class="">Memory per Executor</
th><th width="" class="">Resources Per Executor</th><th width="" class="">Submitted Time</th><th width="" class="">User</th><th width="" class="">State</th><th
width="" class="">Duration</th></thead>
```

Figure 5: Starting the cluster

4.4 Run the Spark Pipeline

```
python ~/spark_clean.py 2>&1 | tee ~/spark_run.log
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ python ~/spark_clean.py 2>&1 | tee ~/spark_run.log
26/10/04 17:31:32 WARN Utils: Your hostname, vishnu-varadhan resolves to a loopback address: 127.0.0.1; using 10.0.2.15 instead (on interface enp0s3)
26/10/04 17:31:32 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/04 17:31:36 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[33.99s] Session started
[97.83s] Ingested: 2964624 trips, 265 zones
[115.02s] Cleansed: 2838136 trips
[166.06s] Joined: 2838136 rows
[220.28s] UDF+group done in 26.91s - 177 result rows
[291.45s] Exported parquet
=== SPARK TOTAL TIME: 291.45s ===
=== SPARK UDF TIME: 26.91s ===
```

Figure 6

```
[33.99s] Session started
[97.83s] Ingested: 2964624 trips, 265 zones
[115.02s] Cleansed: 2838136 trips
[166.06s] Joined: 2838136 rows
[220.28s] UDF+group done in 26.91s - 177 result rows
[291.45s] Exported parquet

=== SPARK TOTAL TIME: 291.45s ===
=== SPARK UDF TIME: 26.91s ===
```

Figure 7: Spark Result

4.5 Stop the Spark Cluster

```
killall -f "org.apache.spark.deploy.master.Master"
killall -f "org.apache.spark.deploy.worker.Worker"
sleep 3
jps
```

5 Ray Cluster Setup (2 CPUs)

5.1 Start the Ray Head Node with Dashboard

```
ray start --head --port=6379 --dashboard-host=0.0.0.0 \
--dashboard-port=8265 --num-cpus=2 --object-store-memory=500000000
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ ray start --head --port=6379 --dashboard-host=0.0.0.0 --dashboard-port=8265 --num-cpus=2 --object-store-memory=500000000
Usage stats collection is enabled. To disable this, add '--disable-usage-stats' to the command that starts the cluster, or run the following command: 'ray disable-usage-stats' before starting the cluster. See https://docs.ray.io/en/master/cluster/usage-stats.html for more details.

Local node IP: 10.0.2.15
2026-10-04 18:39:03,446 WARNING authentication_token_setup.py:73 -- Token authentication is disabled for this Ray cluster. Anyone with network access to the cluster can run arbitrary code and access its data. To enable token authentication, generate a token with 'ray get-auth-token --generate' or set RAY_AUTH_MODE=token. For more information, see https://docs.ray.io/en/latest/ray-security/token-auth.html
2026-10-04 19:00:06,285 ERROR services.py:1431 -- Failed to start the dashboard
2026-10-04 19:00:06,300 ERROR services.py:1456 -- Error should be written to 'dashboard.log' or 'dashboard.err'. We are printing the last 20 lines for you. See 'https://docs.ray.io/en/master/ray-observability/user-guides/configure-logging.html#logging-directory-structure' to find where the log file is.
2026-10-04 19:00:06,325 ERROR services.py:1500 --
The last 20 lines of /tmp/ray/session_2026-10-04-18-39-03_451937_4063/logs/dashboard.log (it contains the error message from the dashboard):

-----
Ray runtime started.
-----

Next steps
Note: The following commands are intended for use on the head node or within the cluster network.

To add another node to this Ray cluster, run
  ray start --address='10.0.2.15:6379'

To connect to this Ray cluster:
  import ray
  ray.init()

To terminate the Ray runtime, run
  ray stop

To view the status of the cluster, use
  ray status
```

Figure 8

5.2 Verify the Cluster

```
sleep 20
ray status
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ ray status
===== Autoscaler status: 2026-10-04 18:01:12.364360 =====
Node status
-----
Active:
  1 node_7eaca82f4875d36349c33e14740ff142e9c656d1770e8ccb2176a175
Pending:
  (no pending nodes)
Recent failures:
  (no failures)

Resources
-----
Total Usage:
  0.0/2.0 CPU
  0B/2.84GiB memory
  0B/476.84MiB object_store_memory

From request_resources:
  (none)
Pending Demands:
  (no resource demands)
```

Figure 9: ray status

```
curl -s http://localhost:8265 | head -20
```

```
(Assignment-3) vishnu@vishnu-varadhan:~$ curl -s http://localhost:8265 | head -20
<!doctype html><html lang="en"><head><meta charset="utf-8"/><link rel="shortcut icon" href="/favicon.ico"/><meta name="viewport" content="width=device-width, in
ltial-scale=1"/><title>Ray Dashboard</title><script defer="defer" src="/static/js/main.ff0259c7.js"></script><link href="/static/css/main.ec1426f1.css" rel="s
tylesheet"></head><body><noscript>You need to enable JavaScript to run this app.</noscript><div id="root"></div></body></html>(Assignment-3) vishnu@vishnu-varadh
```

Figure 10

5.3 Save Evidence Files

```
ray status > ~/ray_status_evidence.txt
curl -s http://localhost:8265 > ~/ray_dashboard_evidence.html
```

5.4 Run the Ray Pipeline

```
cd ~
python ~/ray_clean.py 2>&1 | tee ~/ray_run.log
```

```
[4.11s] Ray initialized
[7.33s] Ingested: 2964624 trips, 265 zones
[27.77s] Cleansed: 2838138 trips
[50.34s] Joined: 2838138 rows
[117.11s] UDF+group done in 62.14s - 177 result rows
[117.33s] Exported parquet

=== RAY TOTAL TIME: 117.33s ===
=== RAY UDF TIME: 62.14s ===
```

Figure 11: Ray result

6 Pipeline Parity Verification

```
python ~/run_all_checks.py 2>&1 | tee ~/parity_check.log
```

This script loads both output parquet files, joins them on (`hour_of_day`, `Borough`), and reports the maximum absolute difference in `avg_speed_mph`.

7 Results

7.1 Execution Time Results

Stage	Spark (s)	Ray (s)	Faster
Init (session/cluster start)	33.99	4.11	Ray
Ingest	63.84	3.22	Ray
Cleanse	17.19	20.44	Spark
Heavy Join	51.04	22.57	Ray
UDF + groupBy	26.91	62.14	Spark
Export (write Parquet)	71.17	0.22	Ray
Total wall-clock	291.45	117.33	Ray (2.48×)

Table 1: Per-stage and total execution time. Ray is 2.48× faster end-to-end.

Interpretation

Ray is dramatically faster on *startup* (4.11 s vs 33.99 s) because it has no JVM boot cost and no cluster master/worker JVM hierarchy. Ray also dominates on *ingest* and *export*, where its object-store backed zero-copy I/O beats Spark’s JVM-serialized shuffle paths. Spark wins narrowly on

the *cleanse* stage (17.19 s vs 20.44 s) due to vectorized DataFrame operations compiled into JVM bytecode.

The overall winner is clearly Ray: **117.33 s vs 291.45 s**, a $2.48\times$ speedup.

7.2 Python UDF Overhead — Deep Dive

The assignment’s central question: *is Ray’s Python-native UDF path faster than Spark’s JVM-bridged path?*

Stage	Spark (s)	Ray (s)
UDF + groupBy	26.91	62.14
Ratio	$1\times$	$2.31\times$ slower

Table 2: UDF stage timing. Spark is $2.31\times$ faster on this dataset.

7.3 Pipeline Parity Verification

Both pipelines produced **177 result rows** (aggregated by `hour_of_day` $\times$ `Borough`). An outer merge across all rows shows:

Metric	Value
Spark rows	177
Ray rows	177
Rows compared (outer join)	201
Max $ \Delta avg_speed_mph $	5.79×10^{-5}
Mean $ \Delta avg_speed_mph $	4.85×10^{-7}

Table 3: Parity verification between Spark and Ray outputs.

The maximum difference across all matched rows is **0.000058 mph**, which is low. **Pipeline parity is verified.**

7.4 Per-Hour Comparison (Sample)

Hour	Spark speed	Ray speed	Match?
0	43.2808	43.2808	
6	82.2574	82.2574	
11	292.0977	292.0977	
15	46.6280	46.6280	
18	279.8766	279.8766	
23	99.8131	99.8131	

Table 4: Sample of per-hour average speeds. All 24 hours match to 4 decimal places.

Criterion	Spark	Ray
Total wall-clock	291.45 s	117.33 s
Cluster startup	33.99 s	4.11 s
Python UDF stage	26.91 s	62.14 s
Architecture footprint	4 JVMs	1 head node
Pipeline parity	Identical (max diff 5.8×10^{-5})	

Table 5

8 AI Usage

- Whenever i got an error or don't have an idea of what to do, i asked AI what to do.
- I did write commands even though sometimes i don't know what it does exactly.